

Integrantes: Mateo Bosch Vesconi

Patron de diseño elegido:

Visitor

Tipo de patron: Comportamiento

Descripción del Problema que resuelve:

En sistemas complejos, a menudo hay una estructura con diferentes tipos de objetos (por ejemplo, distintos roles de usuarios en un centro educativo) y necesitas aplicarles nuevas operaciones frecuentemente. Si agregamos la lógica de estas nuevas operaciones directamente dentro de las clases de cada objeto, el código se "contamina", se vuelve gigante y rompes el principio de que cada clase debe hacer una sola cosa.

Descripción de la Solución:

El patrón Visitor propone separar los algoritmos (las operaciones) de los objetos sobre los que operan. En lugar de que el objeto ejecute la acción por sí mismo, este "acepta" a un objeto externo llamado "Visitante". El objeto le dice al visitante quién es, y el visitante ejecuta la acción correspondiente.

Participantes:

- **Visitor (Visitante):** Una interfaz que declara un método de visita por cada tipo de elemento que existe.
- **ConcreteVisitor (Visitante Concreto):** Quien realmente implementa la acción específica (ej. un GeneradorDeReportesVisitor).
- **Element (Elemento):** La interfaz de los objetos que declara un método accept (aceptar) para recibir al visitante.

- **ConcreteElement (Elemento Concreto):** Las clases reales (ej. Alumno, Profesor) que implementan accept y le dicen al visitante: *"Soy de esta clase, haz tu trabajo en mí"*.

Colaboración:

Como patrón de comportamiento, el Visitor se concentra en la distribución de responsabilidades y en la interacción entre los objetos, definiendo un mecanismo claro de comunicación. El sistema recorre la lista de elementos y a cada uno le pasa el Visitante. El elemento llama al método del visitante correspondiente a su tipo, pasándose a sí mismo como dato.

Consecuencias:

- Beneficios: Permite agregar nuevos comportamientos (visitantes) sin tocar ni modificar el código de las clases originales de los elementos. Agrupa lógicas similares en un solo lugar.
- Impactos: Si el día de mañana agregas un nuevo tipo de elemento (ej. PersonalAdministrativo), tendrás que modificar todos los visitantes existentes para que aprendan a tratar con él.

Ejemplo práctico para el centro educativo:

AlumnoInicial, AlumnoSecundario y Docente son las clases. Quieres generar boletines de calificaciones. En lugar de programar cómo hacer un boletín dentro de la clase de cada persona, creamos un BoletinVisitor. Cuando el visitante pasa por AlumnoInicial, genera un informe conceptual lúdico; cuando pasa por AlumnoSecundario, genera un reporte numérico; y si pasa por Docente, no hace nada o genera un reporte de desempeño.

Bibliografía

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Vargas, C.(s.f). *Unidad 2 - Patrones de Diseño -2006-2* [Apunte de cátedra].

Patrón Visitor (De Comportamiento)

Separando las operaciones de la estructura de datos.



1

¿CUÁL ES EL PROBLEMA?

Agregar nuevas funciones directamente a nuestras clases las vuelve pesadas, difíciles de leer y propensas a errores.

¡Nadie quiere clases mutantes!



2

LA SOLUCIÓN VISITOR

Sacamos la operación de la clase y la ponemos en un "Visitante". La clase solo necesita abrirle la puerta (método `accept`) y el Visitante hace el trabajo pesado.



3

EJEMPLO EN ACCIÓN

1 Estructura:
Tenemos Alumnos y Profesores.



2 Visitante:
Llega el
`GeneradorDeReportesVisitor`.



3 Magia:
El Visitante sabe exactamente qué reporte imprimir para el Alumno y cuál distinto para el Profesor, ¡sin que ellos tengan que programarlo!



4

BENEFICIOS CLAVE



LIMPIEZA:
Clases más pequeñas y enfocadas en sus propios datos.



FLEXIBILIDAD:
Añade nuevas operaciones (visitantes) sin modificar las clases existentes.



CENTRALIZACIÓN:
Toda la lógica de una operación queda agrupada en un solo lugar.